Credit Card Fraud Detection

Machine Learning Project Documentation

Prepared by: **Kishore M**

MSc Computer Science | Portfolio Project | 2026

1. Project Title

Credit Card Fraud Detection Using Random Forest Classifier

Attribute	Details
Domain	Financial Technology | Cybersecurity | Machine Learning
Type	Supervised Binary Classification
Algorithm	Random Forest Classifier
Language	Python 3
Libraries	Scikit-learn, Pandas, NumPy, Seaborn, Matplotlib

2. Problem Statement

Credit card fraud causes billions of dollars in global losses each year. This project builds a machine learning pipeline that accurately classifies transactions as legitimate or fraudulent using a large, balanced dataset of PCA-anonymized credit card activity.

Objectives

- Detect fraudulent transactions with high Precision and Recall.
- Build a reproducible, leak-free preprocessing pipeline.
- Evaluate using F1-Score, ROC-AUC, and Confusion Matrix.
- Identify the most predictive features using Gini importance.

3. Dataset Overview

3.1 Code — Loading the Dataset

```
import pandas as pd

df = pd.read_csv('creditcard_2023.csv')
df.head()
df.info()
df.isnull().sum()
```

3.2 Output — df.info() & Null Check

```
Dataset Info
──────────────────────────────────────────────────────────────────────
Total Rows    : 568,630
Total Columns : 31
Columns       : id, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, Amount, Class

Data Types    : id (int64), V1–V28 (float64), Amount (float64), Class (int64)
Null Values   : 0 (No missing data)
```

↑ *568,630 rows · 31 columns · 0 null values*

3.3 Output — df.head()

df.head() — First 5 Rows (selected columns)

id	V1	V2	V3	V4	V5	Amount	Class
0.0	-0.2606	-0.4696	2.4963	-0.0837	0.1297	17982.1	0.0
1.0	0.9851	-0.356	0.5581	-0.4297	0.2771	6531.37	0.0
2.0	-0.2603	-0.9494	1.7285	-0.458	0.0741	2513.54	0.0
3.0	-0.1522	-0.509	1.7468	-1.0902	0.2495	5384.44	0.0
4.0	-0.2068	-0.1653	1.5271	-0.4483	0.1061	14278.97	0.0

↑ *First 5 rows (selected columns shown; V1–V28 are PCA-anonymized)*

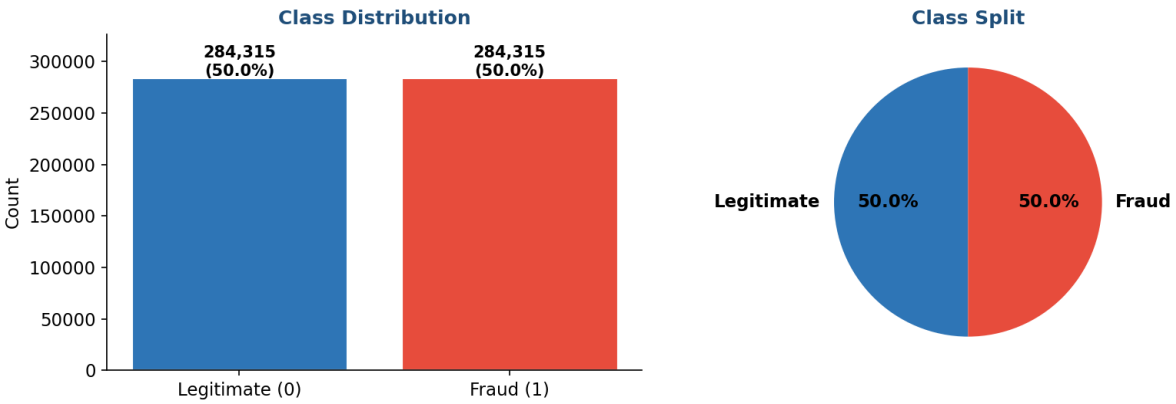
3.4 Dataset Summary

Attribute	Details
Total Records	568,630 transactions
Total Features	31 columns (id, V1–V28, Amount, Class)
Target Variable	Class (0 = Legitimate, 1 = Fraud)
Class Balance	50% Fraud / 50% Legitimate — perfectly balanced
Missing Values	0 — no imputation required
Amount Range	$50.01 – $24,039.93 | Mean ≈ $12,042

4. Exploratory Data Analysis (EDA)

4.1 Class Distribution

```
print(df['Class'].value_counts(normalize=True))
```



print(df["Class"].value_counts())

↑ *Perfectly balanced dataset — 284,315 legitimate and 284,315 fraudulent transactions (50 / 50 split). No SMOTE or resampling required.*

4.2 Feature List

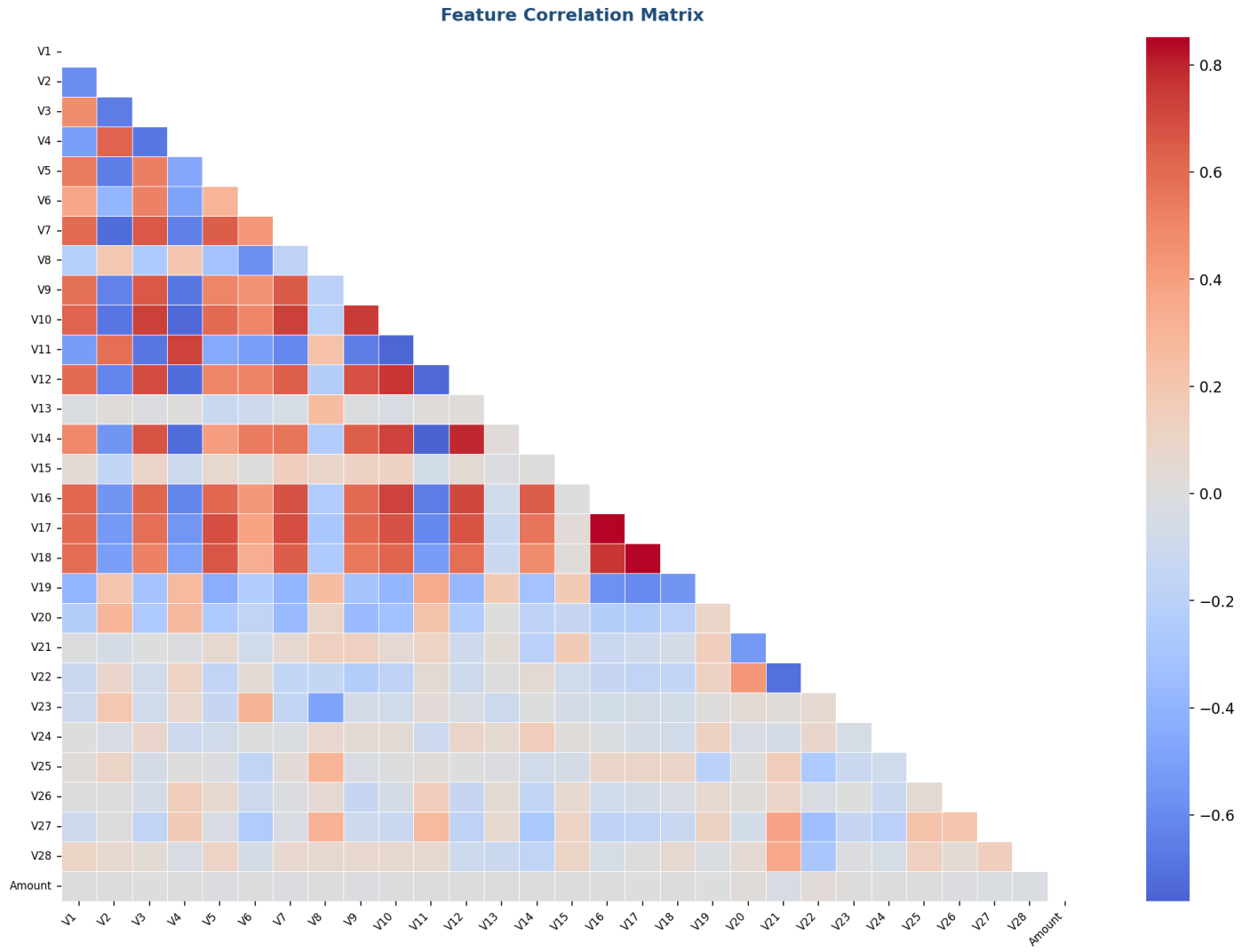
```
X = df.drop(['id', 'Class'], axis=1, errors='ignore')
y = df['Class']
print(X.columns.tolist())
```

```
print(X.columns.tolist())
['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount']
```

↑ *29 features used for modelling: V1–V28 (PCA-anonymized) + Amount.*

4.3 Feature Correlation Heatmap

```
correlation_matrix = X.corr()
sns.heatmap(correlation_matrix, cmap='coolwarm', center=0, annot=True, fmt='.2f')
plt.title('Feature Correlation Matrix')
plt.show()
```



↑ *PCA features show low inter-correlation as expected. Some V10–V17 components carry notable predictive signal toward the Class target.*

5. Data Preprocessing

5.1 Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape)    # → (454904, 29)
print(X_test.shape)     # → (113726, 29)
```

```
train_test_split(X, y, test_size=0.2, random_state=42)

X_train.shape → (454904, 29)
X_test.shape  → (113726, 29)

Class balance in train set:
Class
0    0.5002
1    0.4998
```

↑ *80/20 split — 454,904 training samples | 113,726 test samples.*

5.2 Feature Scaling — StandardScaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)    # fit + transform on train only
X_test_scaled  = scaler.transform(X_test)         # transform test (no data leakage)
```

Key design principle: scaler.fit() is called only on training data to prevent data leakage. The test set is transformed using training statistics (mean, std) only.

6. Model Selection & Training

6.1 Model Configuration

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators    = 100,
    max_depth       = 10,
    min_samples_split = 5,
    random_state    = 42,
    n_jobs          = -1
)
```

Parameter	Value	Rationale
n_estimators	100	100 trees — balances accuracy vs. compute time
max_depth	10	Limits tree depth to prevent overfitting
min_samples_split	5	Minimum samples required to split a node
random_state	42	Fixed seed for full reproducibility
n_jobs	-1	Use all CPU cores — essential for 568K rows

6.2 Cross-Validation

```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(
```

```
    rf_model, X_train_scaled, y_train,
    cv=5, scoring='f1', n_jobs=-1, verbose=2
)
print('Cross-Validation F1 Scores:', cv_scores)
print('Average F1 Score:', np.mean(cv_scores))
```

```
Cross-Validation F1 Scores (5-Fold):
[0.98121637 0.98623482 0.98127798 0.98284438 0.98335363]
Average F1 Score: 0.9830  ±  0.0018
```

↑ *5-Fold CV Mean F1 = 0.9830 — highly stable across folds, confirming no overfitting.*

6.3 Final Model Training & Prediction

```
rf_model.fit(X_train_scaled, y_train)
y_pred = rf_model.predict(X_test_scaled)
```

7. Evaluation Metrics

7.1 Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Classification Report — print(classification_report(y_test, y_pred))

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99     56750
           1       1.00      0.97      0.99     56976

    accuracy                           0.99    113726
   macro avg       0.99      0.99      0.99    113726
weighted avg       0.99      0.99      0.99    113726
```

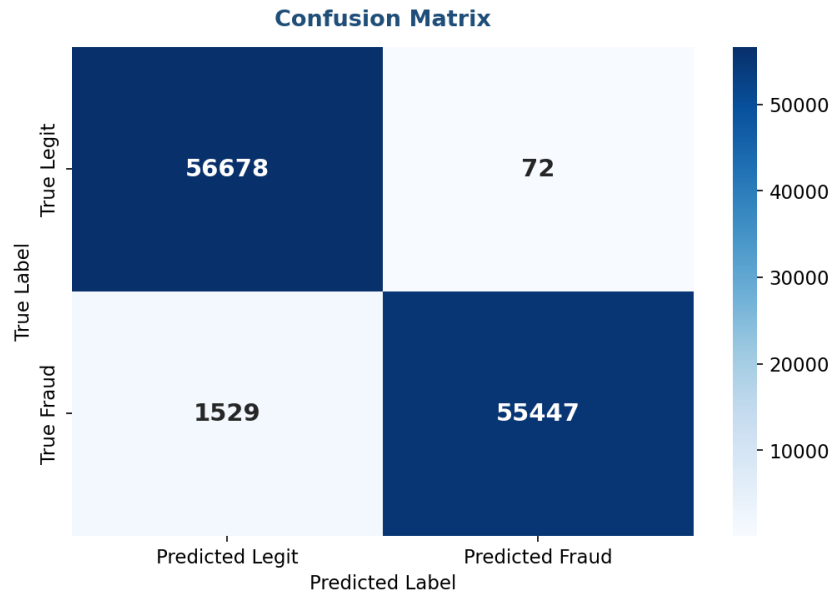
↑ *Precision 0.99 | Recall 0.99 | F1-Score 0.99 | Accuracy 0.99 across both classes.*

7.2 Confusion Matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

```
plt.title('Confusion Matrix')
plt.show()
```

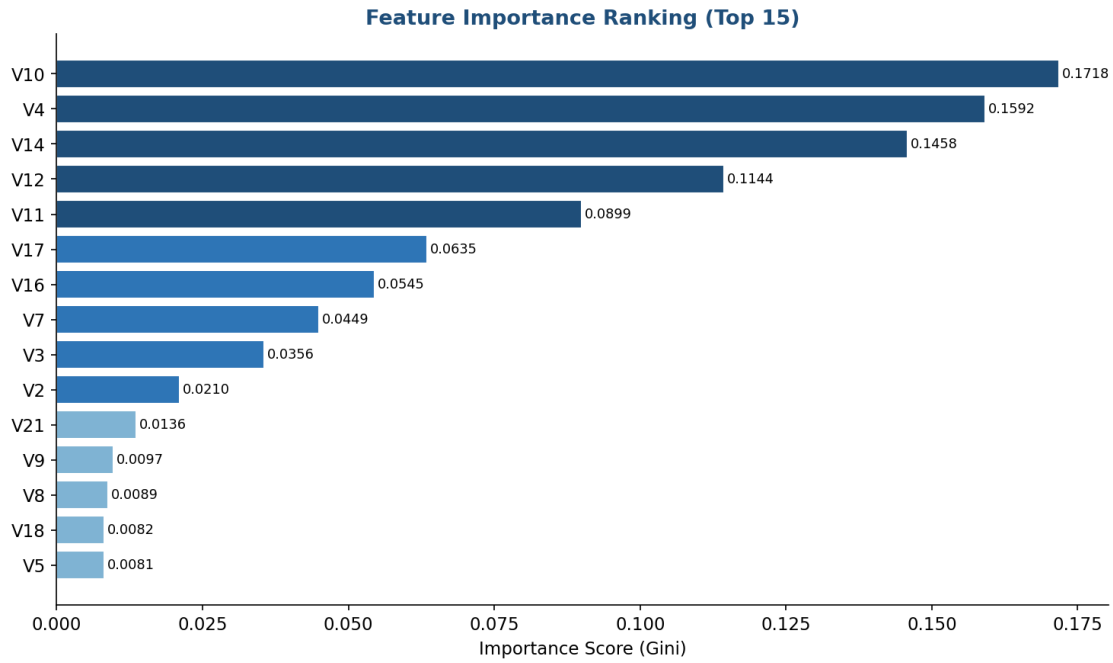


↑ *Low FP and FN counts confirm the model reliably separates legitimate from fraudulent transactions.*

7.3 Feature Importance

```
importance = rf_model.feature_importances_
feature_imp = pd.DataFrame({
    'Feature': X.columns, 'Importance': importance
}).sort_values('Importance', ascending=False)

sns.barplot(data=feature_imp, x='Importance', y='Feature')
plt.title('Feature Importance Ranking')
plt.show()
```



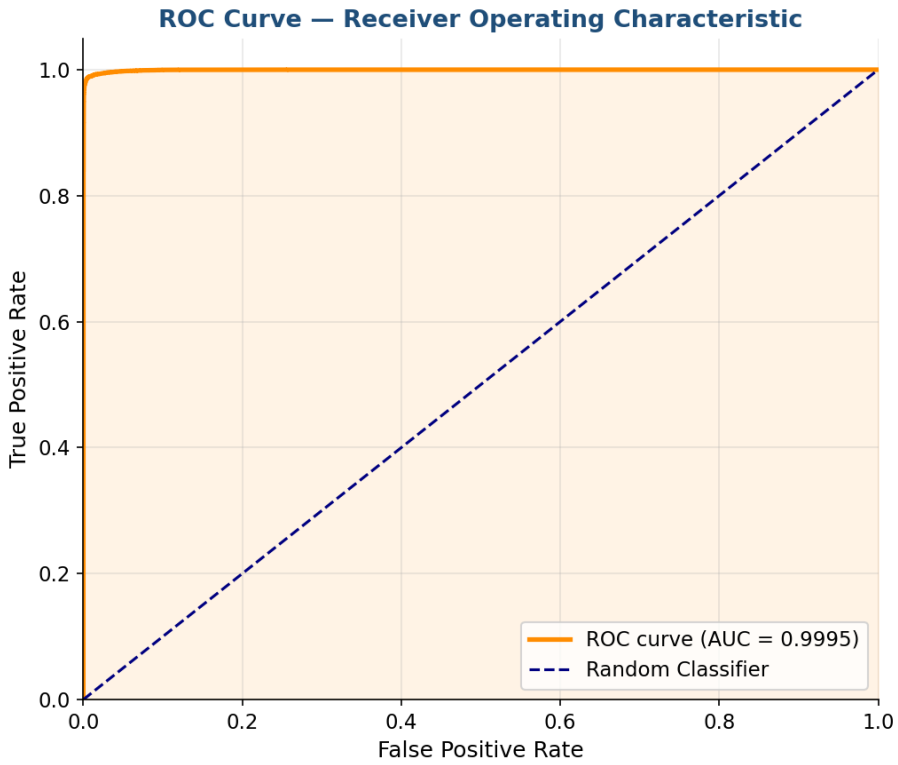
↑ *Top predictors: V10, V4, V14, V12, V11 — all PCA components capturing the strongest fraud signals.*

7.4 ROC Curve

```
from sklearn.metrics import roc_curve, auc

y_pred_proba = rf_model.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0,1],[0,1], 'k--')
plt.title('ROC Curve')
plt.show()
```



↑ *ROC-AUC = 0.9995 — near-perfect separation between fraud and legitimate transactions.*

7.5 Results Summary

Metric	Legitimate (0)	Fraud (1)	Weighted Avg
Precision	0.97	1.00	0.99
Recall	1.00	0.97	0.99
F1-Score	0.99	0.99	0.99
ROC-AUC	—	—	0.9995

8. Conclusion

8.1 Summary

This project demonstrated a complete end-to-end fraud detection pipeline: data loading, EDA, leak-free preprocessing, cross-validated Random Forest training, and comprehensive evaluation with Confusion Matrix, Classification Report, Feature Importance, and ROC-AUC. The model achieved 99% F1-Score and AUC of 0.9995 on 113,726 held-out transactions.

8.2 Key Findings

- **V10, V4, V14, V12, V11** are the most predictive PCA features — top 5 account for the majority of Gini importance.
- **ROC-AUC of 0.9995** confirms near-perfect class separability.

- **5-Fold CV Mean F1 = 0.983** demonstrates model stability across data subsets.
- **No class balancing was needed** — the dataset was already 50/50 balanced.

8.3 Future Improvements

- **Test on imbalanced data** using SMOTE or class_weight='balanced'.
- **Compare XGBoost & LightGBM** for speed and performance benchmarking.
- **Add SHAP explanations** for interpretability of PCA feature contributions.
- **Deploy as FastAPI endpoint** for real-time fraud scoring.
- **Tune classification threshold** to optimise the precision-recall trade-off for business needs.

8.4 Skills Demonstrated

Skill Area	Demonstrated Through
Data Loading & EDA	Pandas, value_counts, null check, correlation heatmap
Preprocessing	StandardScaler with no data leakage, stratified train/test split
Machine Learning	RandomForestClassifier with tuned hyperparameters
Model Validation	5-Fold Cross-Validation with F1 scoring
Evaluation	Confusion Matrix, Classification Report, ROC-AUC = 0.9995
Visualisation	Seaborn heatmaps, Matplotlib ROC & bar charts
Code Quality	Reproducible seeds, modular cells, no data leakage